

The Integrity Stack & Culture Hub

Product Vision: A white-label "Integrity-as-a-Service" (IaaS) platform and discovery directory that uses trust-weighted reviews, verified physical presence, and video UGC to create a non-manipulable authenticity engine for regulated and independent industries.

1. Executive Summary

The Integrity Stack is a decentralized ecosystem designed to amplify the voices of independent operators while maintaining a core "anti-corporate" and "integrity-first" ethos. By transforming static directory data into a living, trust-weighted community record, the system provides a "Semantic Moat" that search engines and AI models prioritize over traditional, ad-driven review platforms.

2. The Culture Hub Ecosystem

The Hub differentiates between community members to ensure quality without creating a pay-to-play hierarchy.

Participant Roles

- **Community Scouts (General Users):** Focus on store/product reviews and "Verified Presence" check-ins via QR codes.
- **Culture Leads (Influencers/Creators):** Focus on high-fidelity content (video tours, blog deep-dives). They provide vetted storytelling that humanizes independent operators.
- **Sovereign Operators (Merchants):** Independent businesses whose rankings are determined by the **IntegrityScore** rather than ad spend.

Non-Gamified Badge System

Badges serve as functional trust signals, automatically authenticated by system logic.

- **Verified Scout:** Granted after 3 unique physical QR check-ins.
 - **Legacy Archivist:** Granted when a creator's content is linked to 5+ Sovereign-tier profiles.
 - **Integrity Anchor:** Triggered by a clean review history (low spam/flag rate) over 6 months.
-

3. The Verified Presence Model (Rotating QR)

To prevent "couch reviews" and fraud, the system anchors digital reviews to physical visits.

The Rotating Logic

- **Digital Displays:** QR codes refresh every 60 seconds using a time-based hash.

- **Static Print (Stickers):** Uses a "Static Seed." The rotation happens in the logic: the app sends [Static ID + User ID + Timestamp + GPS]. The server validates this hash and ensures it hasn't been reused.
- **Verification Gate:** A user is limited to one verified check-in per location every 30 days to prevent engagement stuffing.

4. The Technical Integrity Engine (Backend)

The backend is built as a multi-tenant SaaS that can be toggled between industries (Cannabis, Restaurant, Health).

Data Models (Django Structure)

- **Industry & IndustryRuleDefinition:** Defines specific rules (e.g., Age 19+ for Cannabis, Health Rating for Restaurants).
- **IntegrityToken:** Manages the UUID hashes for QR scans.
- **Checkin:** Records the physical scan with GPS/IP metadata.
- **Review:** Linked 1-to-1 with a Checkin; includes `trust_weight` and `video_asset_id`.

The IntegrityScore Formula

The score is a weighted average: $\Sigma(\text{rating} \times \text{trust_weight}) / \Sigma(\text{trust_weight})$.

- **Base Weight:** 0.5
- **Video Bonus:** +0.2
- **Scout Status Bonus:** +0.3
- **Geo-Deviation Penalty:** -0.4 (if GPS is outside the store radius).

5. White-Label Licensing & Vertical Application

The stack is designed to be licensed to other niches where reviews are easily gamed.

Niche Adaptations

- **Restaurants:** High weight on "Service" and "Independent" status. Video reviews capture the actual meal ("The Dish").
- **Regulated Retail (Liquor/Vape):** Mandatory age-gating and compliance audit trails.
- **Health & Wellness:** Focus on "Professionalism" and verified patient check-ins.

Licensing Tiers

1. **SaaS Module:** API access for existing apps/POS providers to fetch IntegrityScores.
2. **Regional Index:** A turnkey directory (e.g., "The Vancouver Independent Food Index") sold to BIAs or tourism boards.

6. The Technical Moat (SEO & AI Optimization)

To ensure the platform becomes the "Canonical Data Source" for AI and search engines.

The "AI Moat" Checklist

- **Schema Factory:** Dynamic JSON-LD for `schema.org/CannabisStore` or `schema.org/Restaurant`.
- **VideoObject Schema:** Full metadata for Bunny Stream uploads (name, thumbnail, duration).
- **Semantic City Pages:** Non-thin pages featuring regional integrity stats and FAQ schema.
- **AI Discovery:** A public `/.well-known/ai-plugin.json` to allow LLMs to crawl verified data.

7. Development Roadmap

Phase	Milestone	Deliverable
Phase 1	Base Engine	Industry Rule Tables + Multi-tenant API.
Phase 2	Integrity v1	Weighted Review Logic + Fraud Detection.
Phase 3	Presence	QR Token System + Geo-fencing + Mobile Scanner.
Phase 4	Culture Hub	Video UGC (Bunny Stream) + Creator Badge System.
Phase 5	Launch	LegacyLeaf Directory + Restaurant Pilot (VerifiedEats).

8. API Reference (Core Endpoints)

- `POST /tokens/generate`: Generates the QR seed for merchants.
- `POST /checkins/verify`: Validates GPS, Token, and Frequency.
- `POST /reviews/submit`: Submits trust-weighted content (Text + Video).

- GET /scores/{location_id}: Returns the public Integrity Card.

End of Document

To transform the LegacyLeaf "Integrity Stack" into a white-label SaaS product, you are essentially moving from building a **Directory** to building **Integrity-as-a-Service (IaaS)**.

In this model, the directory (like LegacyLeaf or a future "Vancouver Eats" guide) is simply the **presentation layer**, while the **Integrity Engine** handles the heavy lifting of authentication, weighting, and fraud detection.

1. The Portable Architecture: "The Integrity Engine"

To make this licensable, the system must be decoupled from the cannabis-specific UI. The core "Engine" consists of three API-driven modules:

- **Module A: The Presence Validator (QR/Geo):** Manages the generation of "Static Seed" QR codes and validates incoming scans using the **Timestamp + UserID + GPS** logic. It issues a "Proof of Presence" token.
- **Module B: The Weighted Review Processor:** Receives review data and applies the **Trust-Weighting algorithm**. It looks at the User's "Scout" status, the Presence Token, and historical "AuditLog" data to assign a weight (e.g., a verified video review = 1.0, an unverified text review = 0.2).
- **Module C: The IntegrityScore Aggregator:** Calculates the final score based on the blend of weighted reviews, visit-to-review ratios, and "Stability" (detecting sudden rating swings).

2. Vertical Application: The Restaurant Case Study

For the restaurant niche, the "Integrity Stack" solves the primary pain point: **Competitor Sabotage** and **"Paid" Review Padding**.

Feature	LegacyLeaf (Cannabis)	Hospitality/Restaurant White-Label
User Role	Community Scout	Verified Diner / Food Scout

Physical Anchor	In-store Counter Sign	Table-top QR or Receipt Print-out
Trust Weight	Focus on Craft/Sovereign	Focus on "Authentic/Independent" vs. Ghost Kitchens
Video Signal	Product/Store Tour	"The Dish" - 15s verified clip of the actual meal

The "Video + Scan" High-Trust Flow:

1. **Scan:** Diner scans a QR code on the table.
2. **Verify:** The IaaS engine validates they are physically at the restaurant.
3. **Capture:** The app prompts: *"Verified! Share a 15-second clip of your experience to boost this local spot's IntegrityScore."*
4. **Process:** The video is hosted via the **Bunny Stream** integration (as defined in your tech stack), ensuring it's 18+ gated (if bar/liquor) and DRM protected.

3. Implementing Video as a Verification Signal

As noted in the Executive Summary (Page 60), the use of **Bunny Stream** allows for high-performance, low-cost video hosting. In a white-label context:

- **Video = Metadata:** The video isn't just content; it's a "Hard Signal." The backend tags the review with `has_verified_video: true`.
- **Integrity Boost:** A video review automatically overrides "Review Dampening" logic that might otherwise hide a new review during a "Review Burst" event, because a video is significantly harder to fake than text.

4. Commercialization & Licensing Models

Since you are positioning this as a neutral layer, you can license it in two distinct ways:

A. The SaaS "Middle-Ware" (API-First)

- **Target:** Existing POS systems (like Toast or Clover) or niche delivery apps.
- **Function:** They send you their review data; you return an **IntegrityScore** and a "Verified" badge for them to display.
- **Revenue:** Per-API call or per-location monthly fee (\$29/mo per restaurant).

B. The Regional Integrity Index (Turnkey)

- **Target:** Local Business Associations (BIAs), Tourism Boards, or Independent Media Groups.
- **Function:** You provide a "Turnkey Directory" (a clone of the LegacyLeaf frontend) tailored to their niche (e.g., "The Portland Independent Coffee Index").
- **Revenue:** Setup fee + Annual licensing + Data insight reports.

5. Integration with the Current App

To move toward this, the next step in development (Sprint 4) should focus on **Parameterizing the Logic:**

- Instead of hard-coding "Sovereign" tiers, create a `TierDefinition` model.
- Ensure the `IntegrityScore` formula is a standalone function that can accept different "Weights" depending on the industry (e.g., in restaurants, "Service" might weigh higher; in cannabis, "Product Transparency" weighs higher).

Integrity-as-a-Service API documentation

(Swagger/OpenAPI style) that shows how an external restaurant app would "Check-in" a user.

This OpenAPI/Swagger-style documentation defines the Integrity Engine API, the core white-label "brain" that powers the Verified Presence and IntegrityScore features.

This API is designed to be consumed by any front-end (LegacyLeaf, a restaurant app, or a POS system) to validate real-world interactions and generate non-manipulable trust signals.

Integrity Engine API v1.0

Base URL: `https://api.integrity-stack.io/v1`

Authentication: `Bearer <API_KEY>` (Merchant or Platform Key)

1. Generate Physical Token (For Merchants)

POST `/tokens/generate`

Used by a tablet display or merchant dashboard to generate the "seed" for the rotating QR code.

- **Request Body:**
- **JSON**

```
{
```

```
"location_id": "rest_99821",

"rotation_type": "dynamic", // options: "dynamic" (60s) or "static" (print-ready)

"ttl_seconds": 60

}
```

-
-
- Response:
- JSON

```
{

"token_hash": "a1b2c3d4e5f6...",

"qr_string": "https://verify.io/s/rest_99821?h=a1b2c3d4",

"expires_at": "2024-05-20T14:30:01Z"

}
```

-
-

2. Verify Presence (The Check-in)

POST /checkins/verify

The critical endpoint called by the User's App when they scan a QR code. This anchors the digital user to the physical location.

- Request Body:
- JSON

```
{

"user_id": "user_4432",

"token_hash": "a1b2c3d4e5f6...", // Scanned from QR

"location_id": "rest_99821",

"device_metadata": {

  "gps": {"lat": 49.2827, "lng": -123.1207},

  "timestamp": 1716213000,
```

```
"ip_address": "24.110.x.x"
```

```
}
```

```
}
```

-
-
- Logic Gate (Backend):
 1. Validates `token_hash` exists and isn't expired.
 2. Performs Geo-Soft Check: Validates GPS is within 50m of `location_id`.
 3. Performs Frequency Check: Ensures `user_id` hasn't checked into this `location_id` in the last 30 days.
- Response:
- JSON

```
{
```

```
"status": "verified",
```

```
"checkin_id": "chk_88122",
```

```
"trust_weight_bonus": 0.5, // Applied to any subsequent review
```

```
"badge_update": "Verified Scout (2/3 Progress)"
```

```
}
```

```
•
```

```
•
```

3. Submit Trust-Weighted Review

POST `/reviews/submit`

Submits content linked to a verified check-in to maximize the impact on the IntegrityScore.

- Request Body:
- JSON

```
{
```

```
"checkin_id": "chk_88122", // Links to the verification above
```

```
"rating": 5,
```

```
"content_text": "Authentic atmosphere, the best farm-to-table in the city.",
```

```
"video_asset_id": "bunny_vod_9912", // Reference to Bunny Stream upload
```

```
"disclosures": {
```

```
  "incentivized": false,
```

```
  "relationship": "customer"
```

```
}
```

```
}
```

-
-
- Response:
- JSON

```
{
```

```
  "review_id": "rev_7712",
```

```
  "final_trust_weight": 0.95, // Calculated based on user history + video + check-in
```

```
  "impact_on_score": "+0.02"
```

```
}
```

-
-

4. Get IntegrityScore (Public Signal)

GET /scores/{location_id}

Fetches the "Integrity Card" data for a business to be displayed on their profile or website.

- Response:
- JSON

```
{
```

```
  "location_id": "rest_99821",
```

```
  "integrity_score": 8.8,
```

```
  "metrics": {
```

```
    "verified_presence_ratio": 0.75, // 75% of reviews are from verified check-ins
```

```

    "review_stability": "High",    // No sudden rating bursts detected

    "weighted_avg_rating": 4.6,

    "content_richness": 0.82      // High prevalence of video/long-form reviews

},

    "last_audit_date": "2024-05-19T00:00:00Z"

}

```

-
-

How this powers the SaaS Licensing Model

1. **Merchant Branding:** A restaurant (e.g., "The Blue Table") buys a license. They use Endpoint #1 to display a QR code on their receipts.
2. **Zero-UI Integration:** The restaurant doesn't need to build a review site. They just use the Integrity Engine APIs to send data from their own app.
3. **The "Integrity Seal":** They embed the data from Endpoint #4 as a widget on their website. Because it's a "Neutral Third Party" API, users trust that score more than a Yelp rating (which can be manipulated by ads).
4. **Anti-Fraud as a Service:** The engine automatically flags "Review Bursts" (e.g., if 50 unverified reviews appear for a restaurant in 10 minutes) and suppresses them from the IntegrityScore.

Python/Django model structure that would support these API endpoints:

To turn this into a white-label product, the database schema needs to be industry-agnostic. Instead of "Stores," we use "Locations"; instead of "Strains," we use "Offerings."

Below is the Python/Django model structure for the Integrity Engine. This schema supports the rotating tokens, the frequency-gated check-ins, and the trust-weighted reviews.

models.py

Python

```
from django.db import models
```

```
from django.contrib.auth.models import User
```

```
from django.utils import timezone
```

```
from datetime import timedelta
```

```
import uuid
```

```
class Industry(models.TextChoices):
```

```
    CANNABIS = 'CANNABIS', 'Cannabis'
```

```
    RESTAURANT = 'RESTAURANT', 'Restaurant/Hospitality'
```

```
    RETAIL = 'RETAIL', 'Regulated Retail'
```

```
    HEALTH = 'HEALTH', 'Health & Wellness'
```

```
class Location(models.Model):
```

```
    """The business entity being reviewed (Store, Restaurant, Clinic)."""
```

```
    external_id = models.CharField(max_length=100, unique=True) # ID from the client's  
system
```

```
    name = models.CharField(max_length=255)
```

```
    industry = models.CharField(max_length=50, choices=Industry.choices)
```

```
    latitude = models.DecimalField(max_digits=9, decimal_places=6)
```

```
    longitude = models.DecimalField(max_digits=9, decimal_places=6)
```

```
    created_at = models.DateTimeField(auto_now_add=True)
```

```
class IntegrityToken(models.Model):
```

```
    """The dynamic 'Seed' for the Rotating QR code."""
```

```
    location = models.ForeignKey(Location, on_delete=models.CASCADE)
```

```
    token_hash = models.UUIDField(default=uuid.uuid4, unique=True)
```

```
    is_static = models.BooleanField(default=False) # True for printed stickers
```

```
    expires_at = models.DateTimeField()
```

```
    def is_valid(self):
```

```
return timezone.now() < self.expires_at
```

```
class Checkin(models.Model):
```

```
    """A verified physical visit."""
```

```
    user = models.ForeignKey(User, on_delete=models.CASCADE)
```

```
    location = models.ForeignKey(Location, on_delete=models.CASCADE)
```

```
    token = models.ForeignKey(IntegrityToken, on_delete=models.SET_NULL, null=True)
```

```
    # Verification Metadata
```

```
    scan_latitude = models.DecimalField(max_digits=9, decimal_places=6)
```

```
    scan_longitude = models.DecimalField(max_digits=9, decimal_places=6)
```

```
    ip_address = models.GenericIPAddressField()
```

```
    verified_at = models.DateTimeField(auto_now_add=True)
```

```
class Meta:
```

```
    # Enforce the 30-day frequency rule at the DB level
```

```
    unique_together = ('user', 'location')
```

```
    # Note: In production, we'd use a custom constraint to allow
```

```
    # one checkin per 30 days rather than just one ever.
```

```
class Review(models.Model):
```

```
    """The UGC content tied to a verification event."""
```

```
    checkin = models.OneToOneField(Checkin, on_delete=models.CASCADE,  
    related_name='review')
```

```
    rating = models.PositiveSmallIntegerField() # 1-5
```

```
    content = models.TextField()
```

```
    video_url = models.URLField(blank=True, null=True) # Linked to Bunny.net/VOD
```

```
# Weighting Logic
```

```
trust_weight = models.FloatField(default=0.0)
```

```
is_flagged = models.BooleanField(default=False)
```

```
def calculate_weight(self):
```

```
    """
```

```
    The core 'Integrity Engine' Logic:
```

```
    - Start with base weight (0.5)
```

```
    - +0.2 if Video is attached
```

```
    - +0.3 if User has 'Culture Lead' status
```

```
    - -0.4 if GPS coordinates were outside the threshold
```

```
    """
```

```
    weight = 0.5
```

```
    if self.video_url: weight += 0.2
```

```
    # ... logic for user history and geo-validation ...
```

```
    return min(weight, 1.0)
```

```
class LocationIntegrityScore(models.Model):
```

```
    """The final calculated signal for the business."""
```

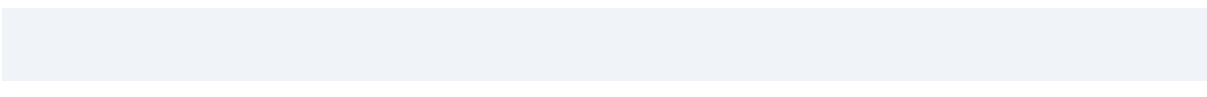
```
    location = models.OneToOneField(Location, on_delete=models.CASCADE)
```

```
    current_score = models.FloatField(default=0.0)
```

```
    verified_visit_count = models.IntegerField(default=0)
```

```
    stability_rating = models.CharField(max_length=20, default="Calculating")
```

```
    last_updated = models.DateTimeField(auto_now=True)
```



Key Logic Implementations

1. The 30-Day "Cool Down"

To prevent users from artificially inflating a restaurant's score by scanning the same QR code every day, you would implement a validation check in the API view:

Python

```
def validate_checkin(user, location):  
  
    thirty_days_ago = timezone.now() - timedelta(days=30)  
  
    recent_checkin = Checkin.objects.filter(  
  
        user=user,  
  
        location=location,  
  
        verified_at__gte=thirty_days_ago  
  
    ).exists()  
  
    if recent_checkin:  
  
        raise ValidationError("You have already verified a visit here this month.")
```

2. Geo-Fencing (The "Soft Check")

When a `Checkin` is posted, the backend compares the `scan_latitude` and `scan_longitude` against the `Location` coordinates. If the distance is greater than 50-100 meters, the check-in is flagged as "Low Trust" or rejected, preventing "couch reviews" from shared screenshots.

3. IntegrityScore Aggregation

The score isn't just a simple average. It's a Periodic Task (using Celery) that runs every 24 hours to recalculate scores using the weighted average of all non-flagged reviews. This allows the system to identify "Review Bursts" (e.g., 100 reviews in 1 hour) and apply a Dampening Factor before they affect the public score.

White-label licensing "Industry Rules" tables to handle different compliance needs (like age-gating for cannabis vs. no age-gating for restaurants):

To scale LegacyLeaf into a white-label "Discovery-as-a-Service" platform, you need a Rule Engine layer in your database. This allows you to swap "Cannabis Rules" (Age 19+, craft-weighted) for "Restaurant Rules" (No age gate, service-weighted) without changing the core code.

1. The "Industry Rules" Schema

This structure uses a Policy-based design. Instead of hardcoding logic, the application queries these tables to determine how a listing should behave and how its score is calculated.

Python

```
from django.db import models
```

```
class Industry(models.Model):
```

```
    """The vertical: e.g., 'Cannabis', 'Hospitality', 'Vape'."""
```

```
    name = models.CharField(max_length=100)
```

```
    slug = models.SlugField(unique=True)
```

```
    description = models.TextField()
```

```
class IndustryRuleDefinition(models.Model):
```

```
    """The 'Blueprint' for a rule. e.g., 'Minimum Age', 'Display Behind Counter'."""
```

```
    industry = models.ForeignKey(Industry, on_delete=models.CASCADE)
```

```
    rule_key = models.CharField(max_length=50) # e.g., "min_age_requirement"
```

```
    data_type = models.CharField(max_length=20) # e.g., "integer", "boolean"
```

```
    is_mandatory = models.BooleanField(default=True)
```

```
class RegionPolicy(models.Model):
```

""Regional overrides for Industry Rules (e.g., Ontario vs Quebec).""

```
rule_def = models.ForeignKey(IndustryRuleDefinition, on_delete=models.CASCADE)
```

```
region_code = models.CharField(max_length=10) # e.g., "ON", "QC", "CA-BC"
```

```
rule_value = models.JSONField() # e.g., {"value": 19} or {"value": 21}
```

class IntegrityWeightingProfile(models.Model):

""Defines how the 'Integrity Score' is calculated for this industry.""

```
industry = models.ForeignKey(Industry, on_delete=models.CASCADE)
```

```
factor_name = models.CharField(max_length=50) # e.g., "Craft/Legacy Status"
```

```
weight_multiplier = models.DecimalField(max_digits=4, decimal_places=2) # e.g., 1.5x
```

```
description = models.CharField(max_length=255)
```

2. Industry Rule Comparison Table

Depending on which "Flavor" of the directory is active, the system enforces these different logic sets:

Rule Feature	Cannabis (LegacyLeaf)	Hospitality (White Label)	Medical/Wellness
Age Gating	Mandatory (19+ / 21+)	Generally None	Optional (18+ for Med)
Review Weighting	High weight on "Legacy Origin"	High weight on "Service/Ambiance"	High weight on "Professionalism"
Verification	Shop License + Identity	Health Inspection + Business License	Medical Credential (HIPAA)

Transparency	Public owner "Trust Score"	Public hygiene/safety ratings	Strict Privacy/Anonymity
Alert Type	"Fresh Drop" (Product Focus)	"Daily Specials" (Service Focus)	"Availability" (Care Focus)

3. Implementation: "Integrity Score" per Industry

The system uses the `IntegrityWeightingProfile` to calculate search rankings dynamically. This prevents a "pay-to-rank" model and ensures the "Independent" spirit is preserved regardless of the industry.

Logic Example:

- Cannabis Listing: $(\text{Owner_Trust} * 1.5) + (\text{Craft_Certification} * 2.0) + (\text{Review_Sentiment} * 0.5)$
- Restaurant Listing: $(\text{Service_Quality} * 1.2) + (\text{Ambiance_Score} * 1.0) + (\text{Review_Sentiment} * 1.0)$

4. Listing Compliance Tracking

To ensure a listing is "Verified," it must pass the rules defined for its industry and region.

Python

```
class ListingCompliance(models.Model):

    """Audit trail for a specific store's compliance."""

    listing = models.ForeignKey('Listing', on_delete=models.CASCADE)

    rule = models.ForeignKey(IndustryRuleDefinition, on_delete=models.CASCADE)

    is_compliant = models.BooleanField(default=False)

    last_checked = models.DateTimeField(auto_now=True)

    evidence_url = models.URLField(blank=True) # Link to photo of age-gate or license
```

Strategic Advantage

By adding these tables, LegacyLeaf is no longer just a cannabis site; it is a Compliance and Integrity Engine. You can pitch this to any regulated industry where "Trust" is harder to find than "Price," allowing you to launch a "LegacyBeer" or

"LegacyVape" directory in days simply by adding new rows to these **Industry** and **Rule** tables.